

Actividad Práctica: UTN RRHH Manager

Triggers, Stored Procedures y Vistas

Este documento describe la actividad práctica sobre la base de datos UTN_RRHH_Manager. El objetivo es poner a prueba los conocimientos sobre Vistas, Procedimientos Almacenados y, fundamentalmente, Triggers complejos con lógica de negocio.

Este documento presenta la actividad práctica sobre la base de datos UTN_RRHH_Manager, diseñada para aplicar y consolidar los conocimientos sobre Vistas, Procedimientos Almacenados y Triggers en SQL Server.

Para ello se provee un sistema que tiene toda la conexión a la base de datos resuelta y los llamados a los objetos necesarios para operar. Es necesario generar los objetos de Base de Datos para que el sistema sea capaz de funcionar completamente.

Requisitos previos para su funcionamiento:

- SQL Server con la base de datos proporcionada en este documento
- Runtime de NET Core 8

Recursos

[Descargar script de base de datos y binarios de aplicación](#)

El sistema simula un módulo de gestión de recursos humanos que administra empleados, feriados y solicitudes de días libres, incorporando reglas de negocio complejas.

NOTA: La aplicación fue creada con Visual Studio Community 2026 y analizada con antivirus. Podés analizar los binarios con VirusTotal o cualquier antivirus para mayor tranquilidad.

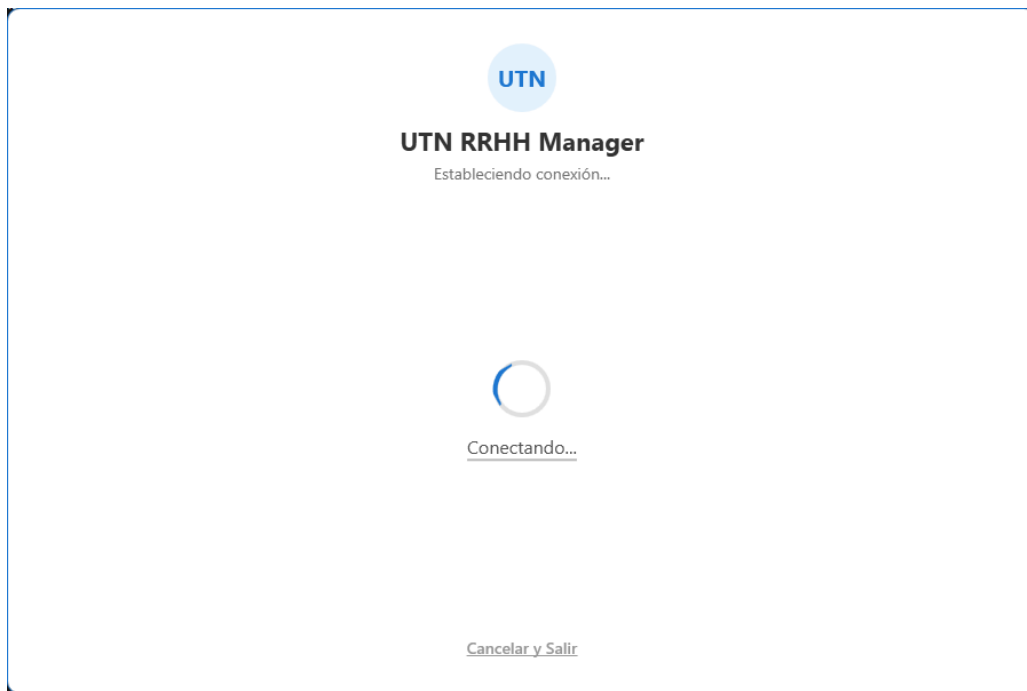


Figura 1. Aplicación conectando a la Base de Datos

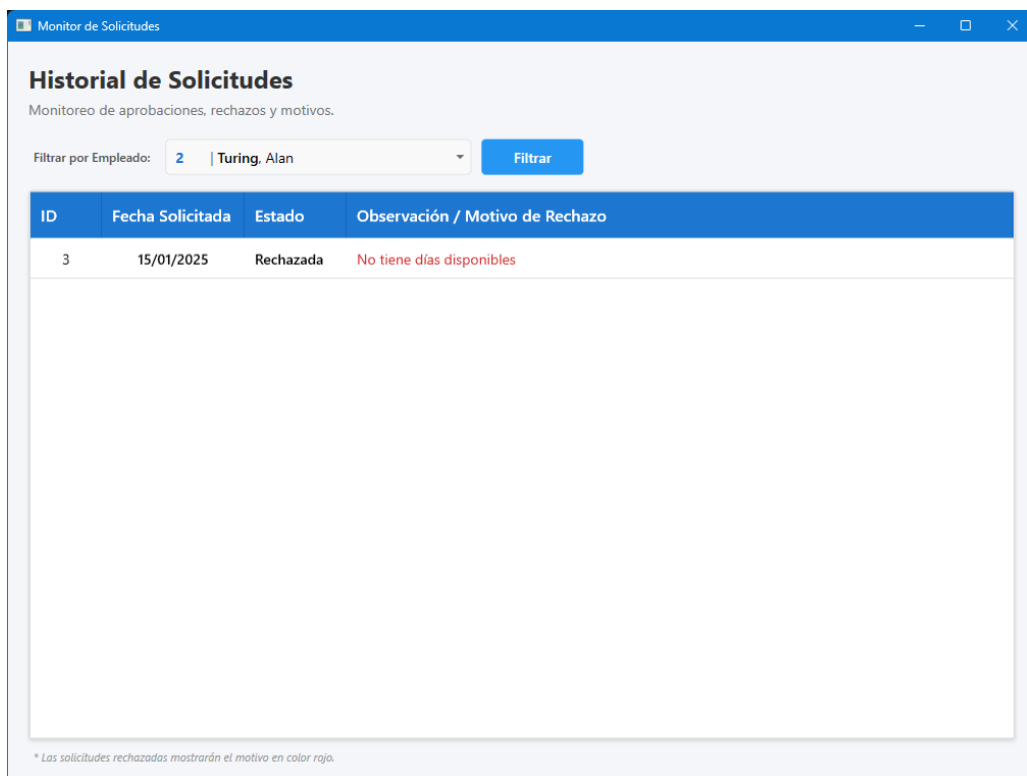


Figura 2. Aplicación mostrando los días libres solicitados por un empleado

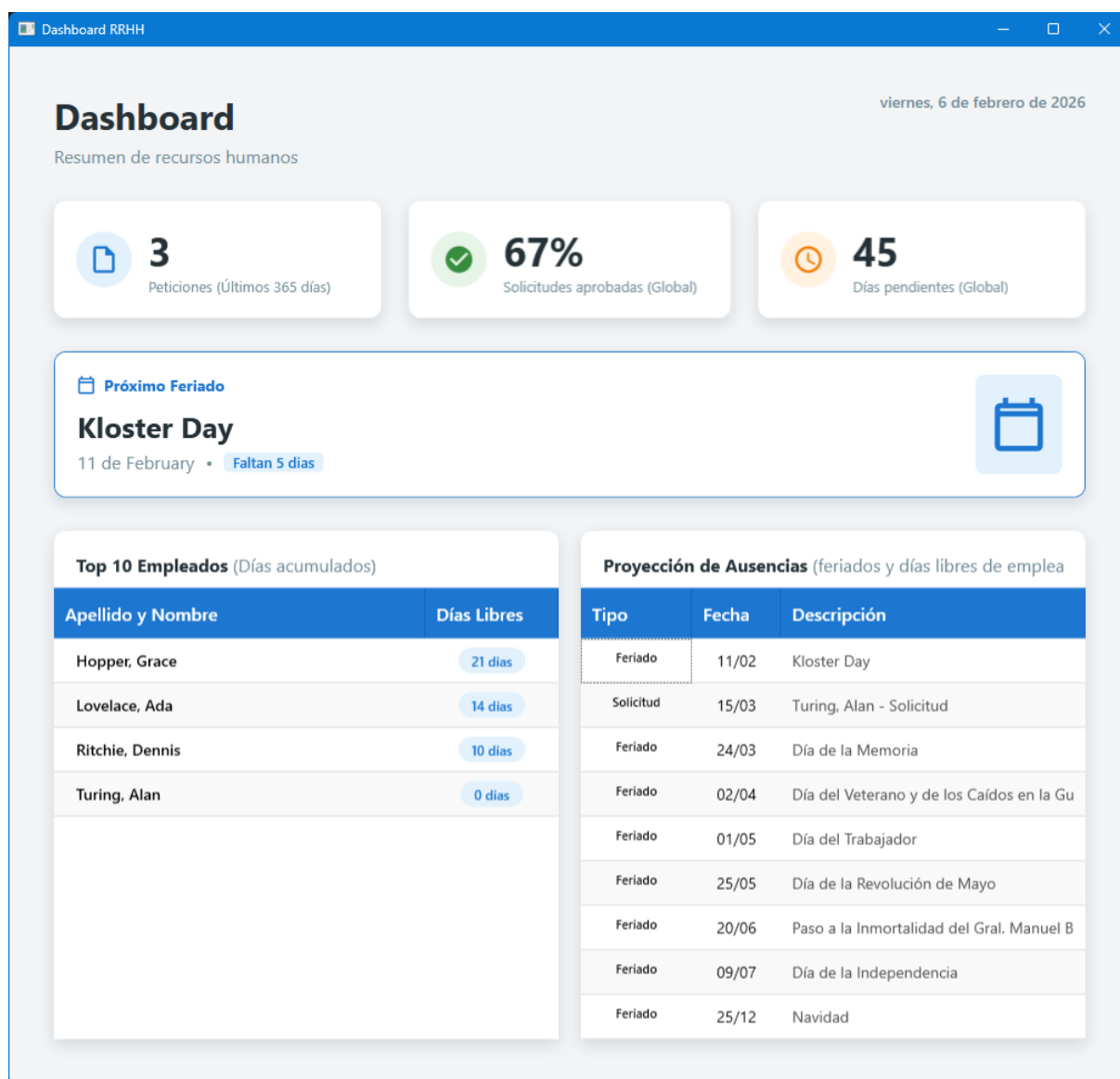


Figura 3. Aplicación mostrando el módulo de Estadísticas

La actividad está estructurada en tres componentes principales: la creación de vistas para simplificar consultas y exponer datos agregados, el desarrollo de procedimientos almacenados para operaciones de alta y consultas estadísticas, y la implementación de un trigger que evalúa automáticamente las solicitudes de días libres según criterios específicos de la organización.

Objetivo de la Actividad

Implementar objetos de base de datos que automaticen la lógica de negocio del sistema de gestión de recursos humanos, garantizando la integridad de los datos y facilitando la integración con la aplicación cliente.

1. Configuración Inicial

1.1. Restauración de la Base de Datos

El primer paso consiste en ejecutar el script SQL que crea la estructura de la base de datos y carga los datos de ejemplo necesarios para las pruebas.

1. Localizar el archivo `Db.sql` en la carpeta de recursos del proyecto.
2. Abrir el archivo en SQL Server Management Studio o el cliente SQL de su preferencia.
3. Ejecutar el script completo para crear la base de datos `UTN_RRHH_Manager`.
4. Verificar que se hayan creado correctamente las tres tablas principales: `Empleados`, `Feriados` y `SolicitudesDiasLibres`.

1.2. Configuración de la Aplicación

La aplicación .NET provista lee la configuración de conexión desde el archivo `UTN.RRHH.Manager.dll.config`. Es necesario configurar la cadena de conexión apropiada según tu entorno.

```
<!-- Agregar dentro de <configuration> -->
<connectionStrings>
  <!-- Autenticación de Windows (Integrated Security) -->
  <add name="DB"
    connectionString="Data Source=YOUR_SERVER_NAME;
                      Initial Catalog=UTN_RRHH_Manager;
                      Integrated Security=True;
                      TrustServerCertificate=True"
    providerName="System.Data.SqlClient" />

  <!-- Autenticación SQL Server (User ID / Password) -->
  <!-- <add name="DB"
    connectionString="Data Source=YOUR_SERVER_NAME;
                      Initial Catalog=UTN_RRHH_Manager;
                      User ID=miUsuario;
                      Password=miPassword;
                      TrustServerCertificate=True"
    providerName="System.Data.SqlClient" /> -->
</connectionStrings>
```

Código 1. Configuración del archivo `App.config`

- Reemplazá `YOUR_SERVER_NAME` por el nombre de tu servidor. Para instancias locales, podés usar `localhost`, . o . `SQLEXPRESS`.

- El parámetro `TrustServerCertificate=True` es necesario en entornos de desarrollo local para evitar problemas con certificados SSL.
- Descomentá la opción de autenticación que corresponda a tu configuración.

Para verificar que la conexión funciona correctamente, ejecutá la aplicación `UTN.RRHH.Manager.exe`. La barra de estado inferior mostrará el estado de la conexión a la base de datos.

2. Estructura de la Base de Datos

La base de datos `UTN_RRHH_Manager` se compone de tres tablas principales que representan las entidades fundamentales del sistema de gestión de recursos humanos.

2.1. Tabla Empleados

Esta tabla almacena la información básica de cada empleado y su saldo actual de días libres disponibles.

Columna	Tipo	Restricción	Descripción
<code>idempleado</code>	INT	PK, IDENTITY(1,1)	Identificador único
<code>apellidos</code>	VARCHAR(100)	NOT NULL	Apellidos del empleado
<code>nombres</code>	VARCHAR(100)	NOT NULL	Nombres del empleado
<code>diaslibres</code>	INT	NOT NULL, DEFAULT 0	Saldo de días disponibles

Tabla 1. Estructura de la tabla Empleados

La columna `diaslibres` representa el stock de días libres que el empleado puede solicitar. Este valor se decrementa automáticamente cuando se aprueba una solicitud.

2.2. Tabla Feriados

Contiene el calendario de feriados de la organización, utilizado tanto para consultas informativas como para validaciones en las reglas de negocio.

Columna	Tipo	Restricción	Descripción
fecha	DATE	PK	Fecha del feriado
descripcion	VARCHAR(200)	NOT NULL	Descripción del feriado

Tabla 2. Estructura de la tabla Feriados

2.3. Tabla SolicitudesDiasLibres

Registra todas las solicitudes de días libres realizadas por los empleados, incluyendo su estado de aprobación y, en caso de rechazo, la razón correspondiente.

Columna	Tipo	Restricción	Descripción
idsolicitud	INT	PK, IDENTITY(1,1)	Identificador único
idempleado	INT	FK, NOT NULL	Referencia al empleado
fechadialibre	DATE	NOT NULL	Fecha solicitada
aprobada	BIT	DEFAULT 0	Estado de aprobación
razonrechazo	VARCHAR(MAX)	NULL	Motivo del rechazo

Tabla 3. Estructura de la tabla SolicitudesDiasLibres

La tabla incluye una clave foránea hacia Empleados mediante la constraint FK_Solicitudes_Empleados.

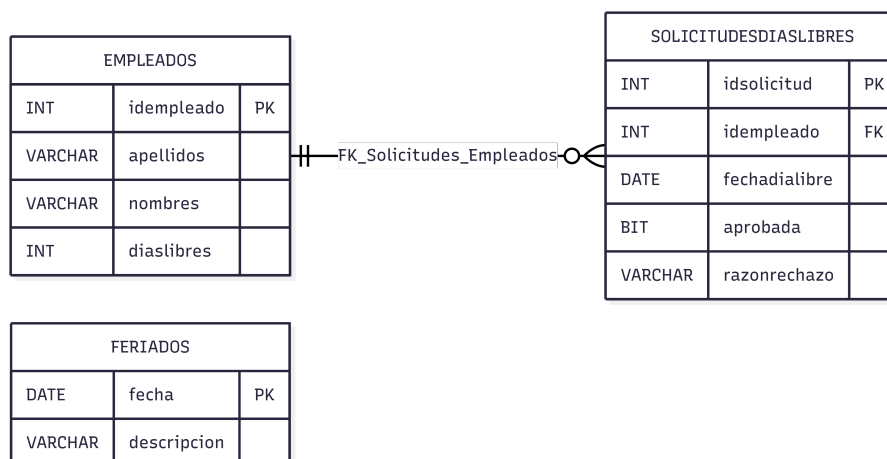


Figura 4. Diagrama de Entidad Relación de la Base de Datos del ejercicio

3. Vistas (VIEW)

Las vistas simplifican las consultas desde la aplicación y permiten exponer columnas calculadas o datos agregados de manera transparente. A continuación se detallan las tres vistas requeridas para esta actividad.

3.1. Vista VW_Feriados

Esta vista debe devolver todos los feriados registrados en el sistema, junto con el cálculo de los días que faltan para cada uno.

Columnas Requeridas

La vista debe retornar exactamente las siguientes columnas con los nombres y tipos especificados:

- fecha (DATE): La fecha del feriado.
- descripcion (VARCHAR(200)): La descripción del feriado.
- diasparaferiado (INT): Cantidad de días que faltan para el feriado, calculado como la diferencia entre la fecha del feriado y la fecha actual.

La aplicación consume esta vista en el módulo de Listado de Feriados y en el Dashboard de Estadísticas, aplicando un ordenamiento ascendente por fecha.

3.2. Vista VW_TopEmpleados

Esta vista proporciona un listado consolidado de empleados con sus nombres completos y saldo de días libres disponibles.

Columnas Requeridas

La vista debe retornar exactamente las siguientes columnas con los nombres y tipos especificados:

- nombrecompleto (VARCHAR): Concatenación del apellido y nombre del empleado, separados por coma y espacio.
- diaslibres (INT): Saldo actual de días disponibles.

La aplicación ordena los resultados de manera descendente por cantidad de días libres y utiliza esta vista en el Dashboard de Estadísticas para mostrar el ranking de empleados.

3.3. Vista VW_Ausencias

Esta vista unifica en una sola consulta tanto los feriados como las solicitudes de días libres aprobadas, facilitando la visualización del calendario de ausencias de la organización.

Columnas Requeridas

La vista debe retornar exactamente las siguientes columnas con los nombres y tipos especificados:

- tipo (VARCHAR(10)): Debe contener el literal 'Feriado' para registros de la tabla Feriados, o 'Solicitud' para días libres aprobados.
- fecha (DATE): La fecha del evento (feriado o solicitud).
- descripcion (VARCHAR(150)): Para feriados, la descripción del mismo; para solicitudes, la concatenación del apellido y nombre del empleado que solicitó el día libre.

La aplicación consume esta vista en el Dashboard de Estadísticas y aplica un ordenamiento ascendente por fecha. Es importante considerar únicamente las solicitudes aprobadas (aprobada = 1) al momento de construir la unión.

Uso de UNION

Para combinar datos de diferentes fuentes en una sola vista, considerará utilizar el operador UNION ALL. Asegurate de que ambas consultas retornen el mismo número de columnas con tipos de datos compatibles.

4. Procedimientos Almacenados (Stored Procedures)

Los procedimientos almacenados encapsulan la lógica de negocio y las operaciones CRUD, facilitando el mantenimiento y mejorando la seguridad de la aplicación.

4.1. Procedimiento SP_AltaEmpleado

Este procedimiento permite registrar un nuevo empleado en el sistema.

Parámetros Requeridos

El procedimiento debe recibir exactamente los siguientes parámetros con los nombres y tipos especificados:

- @apellidos (VARCHAR(100)): Apellidos del empleado.

- @nombres (VARCHAR(100)): Nombres del empleado.
- @diaslibres (INT): Saldo inicial de días libres. Valor por defecto: 0.

El procedimiento debe insertar un nuevo registro en la tabla Empleados y no debe retornar ningún conjunto de resultados mediante SELECT. La aplicación utiliza ExecuteNonQuery para invocarlo, por lo que solo evalúa el número de filas afectadas.

Manejo de Errores

Si es necesario controlar los mensajes de error que muestra la aplicación, utilízalo RAISERROR con mensajes descriptivos. La aplicación captura y muestra el Message de la excepción generada.

El procedimiento es consumido por el módulo de Empleados en la funcionalidad de Alta de Empleado.

4.2. Procedimiento SP_AltaFeriado

Este procedimiento permite registrar un nuevo feriado en el calendario del sistema.

Parámetros Requeridos

El procedimiento debe recibir exactamente los siguientes parámetros con los nombres y tipos especificados:

- @fecha (DATE): Fecha del feriado.
- @descripcion (VARCHAR(200)): Descripción del feriado.

El procedimiento debe insertar un nuevo registro en la tabla Feriados. Es importante implementar validaciones de negocio, como por ejemplo evitar que se registren feriados con fechas pasadas.

Aunque algunas validaciones podrían realizarse en la aplicación cliente, se implementan intencionalmente en la base de datos para reforzar las reglas de negocio a nivel de datos.

4.3. Procedimiento SP_EstadisticasGenerales

Este procedimiento calcula y retorna métricas agregadas para el Dashboard de Estadísticas.

Columnas Retornadas

El procedimiento no recibe parámetros y debe retornar mediante SELECT una única fila con las siguientes columnas:

- **TotalSolicitudes (INT):** Conteo total de solicitudes registradas en los últimos 365 días.
- **PorcentajeAprobacion (DECIMAL):** Proporción de solicitudes aprobadas respecto al total, expresada como decimal.
- **DiasAcumulados (INT):** Suma total de días libres disponibles de todos los empleados.

Para el cálculo del PorcentajeAprobacion, tené en cuenta que si no hay solicitudes en el período, el procedimiento debe manejar adecuadamente la división por cero, retornando 0 o NULL según corresponda.

5. Trigger de Validación de Solicitudes

El componente más crítico de esta actividad es el desarrollo de un trigger que evalúe automáticamente cada solicitud de día libre según las reglas de negocio establecidas por la organización.

5.1. Descripción General

El trigger debe ejecutarse automáticamente después de cada inserción en la tabla `SolicitudesDiasLibres` (AFTER INSERT). Su función es verificar si la solicitud cumple con todas las condiciones necesarias para ser aprobada, o en caso contrario, registrar el motivo específico del rechazo.

Reglas de Negocio

El trigger debe evaluar secuencialmente las siguientes cuatro condiciones para cada solicitud:

1. **Stock de Días:** Verificar que el empleado tenga saldo disponible (`diaslibres > 0`).
2. **Regla de Consecutivos:** Verificar que el empleado no tenga un día libre aprobado el día inmediatamente anterior a la fecha solicitada.
3. **Regla de Feriados:** Verificar que el día inmediatamente anterior a la fecha solicitada no sea un feriado registrado, evitando extensiones artificiales de fines de semana largos.

5.2. Escenario A: Solicitud Aprobada

Si la solicitud cumple con las cuatro reglas establecidas, el trigger debe ejecutar las siguientes acciones:

Acciones para Aprobación

1. Actualizar el registro recién insertado en `SolicitudesDiasLibres`, estableciendo `aprobada = 1`.
2. Descontar 1 día del campo `diaslibres` en la tabla `Empleados` para el empleado correspondiente.
3. El campo `razonrechazo` debe permanecer en `NULL`.

5.3. Escenario B: Solicitud Rechazada

Si la solicitud no cumple con al menos una de las reglas, el trigger debe ejecutar las siguientes acciones:

Acciones para Rechazo

1. Actualizar el registro recién insertado en `SolicitudesDiasLibres`, estableciendo `aprobada = 0`.
2. Escribir en el campo `razonrechazo` el motivo específico del rechazo. El mensaje debe ser descriptivo y ayudar al empleado a comprender por qué no se aprobó su solicitud.
3. **No descontar** días del stock del empleado.

Ejemplos de mensajes de rechazo apropiados:

- Rechazada: no posee días libres disponibles.
- Rechazada: no puede solicitar días consecutivos (día anterior o posterior ya aprobado).
- Rechazada: no puede ser consecutivo al feriado Carnaval - 17/02/2026.

Validación de Fechas

Para verificar si el día anterior a la fecha solicitada es un feriado o tiene una solicitud aprobada, utilizá funciones de fecha como `DATEADD(DAY, -1, @FechaSolicitada)` y consultá las tablas correspondientes.

Orden de Evaluación

Es importante evaluar las reglas en el orden especificado. La primera condición que falle debe detener la evaluación y generar el rechazo correspondiente. Esto garantiza que el mensaje de rechazo sea específico y útil para el usuario.

6. Verificación y Pruebas

Una vez implementadas las vistas, procedimientos almacenados y el trigger, es fundamental realizar pruebas exhaustivas para verificar el correcto funcionamiento del sistema.

6.1. Pruebas de Vistas

Ejecutá consultas directas sobre cada vista para verificar que retornan las columnas esperadas con los tipos de datos correctos:

```
-- Verificar VW_Feriados
SELECT * FROM VW_Feriados;

-- Verificar VW_TopEmpleados
SELECT * FROM VW_TopEmpleados;

-- Verificar VW_Ausencias
SELECT * FROM VW_Ausencias;
```

Código 2. Pruebas de vistas

6.2. Pruebas de Procedimientos Almacenados

Ejecutá cada procedimiento con datos de prueba para verificar su comportamiento:

```
-- Probar alta de empleado
EXEC SP_AltaEmpleado
    @apellidos = 'González',
    @nombres = 'María Laura',
    @diaslibres = 10;

-- Probar alta de feriado
EXEC SP_AltaFeriado
    @fecha = '2026-05-01',
    @descripcion = 'Día del Trabajador';

-- Probar estadísticas generales
```

```
EXEC SP_EstadisticasGenerales;
```

Código 3. Pruebas de procedimientos almacenados

6.3. Pruebas del Trigger

El trigger es el componente más complejo y requiere pruebas detalladas para cada escenario posible:

```
-- Caso 1: Empleado sin días disponibles
INSERT INTO SolicitudesDiasLibres (idempleado, fechadialibre)
VALUES (1, '2026-03-15');

-- Verificar que se rechazó y el motivo
SELECT * FROM SolicitudesDiasLibres
WHERE idsolicitud = SCOPE_IDENTITY();

-- Caso 2: Solicitud de día consecutivo a feriado
INSERT INTO SolicitudesDiasLibres (idempleado, fechadialibre)
VALUES (2, '2026-02-18'); -- Día siguiente a Carnaval

-- Caso 3: Solicitud válida que debe aprobarse
INSERT INTO SolicitudesDiasLibres (idempleado, fechadialibre)
VALUES (3, '2026-04-10');

-- Verificar que se aprobó y se descontó el día
SELECT e.nombres, e.apellidos, e.diaslibres,
       s.aprobada, s.razonrechazo
FROM Empleados e
INNER JOIN SolicitudesDiasLibres s ON e.idempleado = s.idempleado
WHERE s.idsolicitud = SCOPE_IDENTITY();
```

Código 4. Casos de prueba para el trigger

6.4. Integración con la Aplicación

Una vez verificado el funcionamiento mediante queries directas, ejecutá la aplicación `UTN.RRHH.Manager.exe` y probá cada módulo:

1. **Módulo de Empleados:** Probá el alta de nuevos empleados y verificá que se visualicen correctamente.
2. **Módulo de Feriados:** Agregá nuevos feriados y verificá que aparezcan en el listado con los días calculados correctamente.

3. **Módulo de Solicitudes:** Creá solicitudes de días libres y verificá que el trigger evalúe correctamente cada caso, mostrando aprobaciones o rechazos con sus motivos.
4. **Dashboard de Estadísticas:** Verificá que las métricas agregadas se calculen correctamente y que las vistas muestren información consistente.

Estado de Conexión

La barra de estado en la parte inferior de la aplicación indica el estado de conexión a la base de datos. Si no se conecta correctamente, revisá la configuración del App.config.

7. Conclusión

Esta actividad práctica integra los conceptos fundamentales de programación de base de datos en SQL Server, combinando vistas para simplificar consultas, procedimientos almacenados para encapsular operaciones y triggers para automatizar la lógica de negocio.

La implementación correcta de estos componentes demuestra la capacidad para diseñar soluciones que garanticen la integridad de los datos, automaticen procesos y faciliten el mantenimiento del sistema. Las reglas de negocio implementadas en el trigger ejemplifican cómo trasladar requerimientos organizacionales a código ejecutable que opera de manera transparente para los usuarios.